601.433/633 Intro Algorithms
Recitation 6

Fall 2026

Maximum Weight Independent Set in a Tree

You are given an undirected tree $T = (V, E)$ with $n \geq 1$ vertices and a real weight $w(v)$ on each vertex (weights may be negative). Root the tree at any vertex r. An *independent set* is a subset $S \subseteq V$ containing no two adjacent vertices. Our goal is to find an independent set of maximum total weight. Design a dynamic programming algorithm, and prove its correctness and running time.

Solution:

Let T_v be the subtree rooted at v, with children $\mathrm{ch}(v)$. Define $OPT_{in}(v)$ as the maximum weight of an independent set of T_v that *includes* v, and $OPT_{out}(v)$ as the maximum weight of an independent set of T_v that *excludes* v.

Theorem 1 (**Optimal substructure theorem**). *For every vertex v,*

$$\boxed{\begin{aligned} OPT_{in}(v) &= w(v) + \sum_{u \in \mathrm{ch}(v)} OPT_{out}(u), \\ OPT_{out}(v) &= \sum_{u \in \mathrm{ch}(v)} \max\{OPT_{in}(u), OPT_{out}(u)\}. \end{aligned}}$$

Proof. We first record the structural property that makes the argument work: the vertex sets of the child subtrees T_u, for $u \in \mathrm{ch}(v)$, are pairwise disjoint, and there are no edges between distinct child subtrees. Within T_v, the only edge joining T_u to a vertex outside T_u is $\{v, u\}$.

Case 1: v is included. Let S be a maximum-weight independent set of T_v subject to $v \in S$. For each child u of v, put $S_u = S \cap V(T_u)$. Since S is independent and $v \in S$, we have $u \notin S_u$; hence S_u is a feasible independent set of T_u that excludes u. We claim that each S_u is optimal, i.e., it has maximum weight. Suppose not. Then for some child u, there is an independent set $S'_u \subseteq V(T_u)$ with $u \notin S'_u$ and

$$w(S'_u) > w(S_u), \qquad \text{where } w(A) := \sum_{x \in A} w(x).$$

Replace the restriction of S to T_u by S'_u to get:

$$S' = (S \setminus S_u) \cup S'_u.$$

This replacement preserves independence: S'_u is internally independent, there are no edges from T_u to another child subtree, and the only possible edge from T_u to the included vertex v is $\{v, u\}$, whose endpoint u is excluded from S'_u. Moreover $v \in S'$, and

$$w(S') = w(S) - w(S_u) + w(S'_u) > w(S),$$

contradicting the optimality of S. Thus $w(S_u) = OPT_{out}(u)$ for every child u. Since $S = \{v\} \cup \bigcup_{u \in \mathrm{ch}(v)} S_u$,

$$OPT_{in}(v) = w(S) = w(v) + \sum_{u \in \mathrm{ch}(v)} OPT_{out}(u).$$

Case 2: v is excluded. Let S be a maximum-weight independent set of T_v subject to $v \notin S$, and again define $S_u = S \cap V(T_u)$ for every child u. Here S_u may contain u, because v is excluded. As before, we claim that each S_u is a maximum-weight independent set of T_u without any constraint on u. Otherwise, for some child u there exists an independent set $S'_u \subseteq V(T_u)$ with $w(S'_u) > w(S_u)$. The replacement $S' = (S \setminus S_u) \cup S'_u$ remains independent: there are no edges between different child subtrees, and the only edge from T_u to v causes no conflict because $v \notin S'$. Thus it excludes v and has greater weight than S, a contradiction.

Therefore each S_u has weight $\max\{OPT_{in}(u), OPT_{out}(u)\}$, according to whether or not an optimal solution in T_u contains u. As the child subtrees partition $T_v \setminus \{v\}$,

$$OPT_{out}(v) = w(S) = \sum_{u \in \mathrm{ch}(v)} \max\{OPT_{in}(u), OPT_{out}(u)\}.$$

□

Algorithm. The input tree is given by undirected adjacency lists Adj[v]. We first root the tree at an arbitrary vertex r, and then compute a postorder traversal using DFS. The DFS procedure constructs the children lists children[v] by recording each newly discovered vertex as a child of the vertex from which it was discovered. It also constructs a postorder sequence P by appending each vertex after recursively exploring all its children.

```
PostorderDFS(v) {
    visited[v] = True;
    for each u in Adj[v] {
        if visited[u] == False then {
            append u to children[v];
            PostorderDFS(u);
        }
    }
    append v to P;
}
```

Next, we compute the values of two DP tables, IN and OUT, in postorder to ensure that a vertex is visited after all its children:

```
MWIS(T) {
    choose an arbitrary root r;
```

```
    for each v in V {
        visited[v] = False;
        children[v] = empty list;
    }
    P = empty list;
    PostorderDFS(r);

    for each v in P {
        IN[v] = w[v];
        OUT[v] = 0;
        for each u in children[v] {
            IN[v] = IN[v] + OUT[u];
            OUT[v] = OUT[v] + max(IN[u], OUT[u]);
        }
    }
    return max(IN[r], OUT[r]);
}
```

Theorem 2 (**Algorithm Correctness**). *The algorithm computes* $\mathrm{IN}[v] = OPT_{in}(v)$ *and* $\mathrm{OUT}[v] = OPT_{out}(v)$ *for all vertices* v*. Hence* MWIS *returns the maximum weight of an independent set in* T*.*

Proof. We first establish that for a given rooted tree, the preprocessing produces valid children lists for all vertices, and also a valid postorder. The subroutine PostorderDFS(r) marks a vertex visited before exploring its neighbors, so no vertex is visited twice. Since T is connected, every vertex is reached. When DFS first discovers an unvisited vertex u from v, it records u in children[v]. Since T is a tree, every non-root vertex is discovered exactly once from its unique parent in the rooted tree, and is added to the children list of the said parent. Further, a non-root vertex's parent is already visited, and thus is not included in its children list. Finally, PostorderDFS(v) appends v to P only after every recursive child call returns. Thus, every child precedes its parent in P.

We now prove the DP claim by *strong induction on* $|T_v|$. If v is a leaf, children$[v] = \emptyset$, so the DP loop assigns $\mathrm{IN}[v] = w(v)$ and $\mathrm{OUT}[v] = 0$. These values are correct: the only independent set containing v is $\{v\}$, and the only independent set excluding it is the empty set.

For the inductive step, let v be a non-leaf and suppose the claim holds for every subtree with fewer vertices than T_v. For each child u of v, we have $|T_u| < |T_v|$, and the postorder property guarantees that u has already been processed when the algorithm reaches v. Hence the induction hypothesis gives $\mathrm{IN}[u] = OPT_{in}(u)$ and $\mathrm{OUT}[u] = OPT_{out}(u)$. The inner loop visits every child exactly once, so the computed values are

$$\mathrm{IN}[v] = w(v) + \sum_{u \in \mathrm{children}[v]} OPT_{out}(u) = OPT_{in}(v),$$

$$\mathrm{OUT}[v] = \sum_{u \in \mathrm{children}[v]} \max\{OPT_{in}(u), OPT_{out}(u)\} = OPT_{out}(v).$$

The last equalities follow from the optimal-substructure theorem (Theorem 1). This completes the induction. Since every independent set of T either contains or excludes r, the returned maximum is optimal. □

Running time analysis. We will analyze the running times of the DFS procedure and the DP algorithm separately, and then add them:

DFS postorder: Initializing visited[v] and children[v] takes $\Theta(1)$ time at each of the n vertices, hence $\Theta(n)$ altogether. Each vertex is visited by DFS exactly once. Because a tree has $|E| = n - 1$ edges, the for loop inspects exactly

$$\sum_{v \in V} \deg(v) = 2|E| = 2(n-1)$$

adjacency-list entries across *all* iterations. Each inspection performs a constant-time visited test, and exactly $n-1$ successful tests append a child and initiate a recursive call. Each of the n vertices is appended to P exactly once, after its descendants. Consequently, the DFS postorder processing takes time $\Theta(n) + \Theta\big(2(n-1)\big) + \Theta(n) = \Theta(n)$.

Dynamic programming: The outer loop iterates exactly n times and performs two constant-time initializations in each iteration, costing $\Theta(n)$ altogether. The inner loop at vertex v runs exactly $|\text{children}[v]|$ times. Since every non-root vertex appears in precisely one child list,

$$\sum_{v \in V} |\text{children}[v]| = n - 1,$$

which tells us that the total number of inner loop iterations across *all* outer loop iterations is exactly $n-1$. Every inner loop iteration performs a constant number of additions and maximums, so the total time taken for all inner loop iterations is $\Theta(n-1)$. Thus, the DP algorithm runs in time $\Theta(n) + \Theta(n-1) = \Theta(n)$.

Therefore, the total running time is $\Theta(n) + \Theta(n) = \boxed{\Theta(n)}$.

Recovering an optimal independent set. MWIS returns the optimum weight. For the actual independent set, our algorithm runs a second pass after filling in the two tables. It constructs the set by descending from the root r, including r if $\text{IN}[r] \geq \text{OUT}[r]$. When a vertex is included, the algorithm forces all its children to be excluded. Otherwise, it chooses the better state of each child (ties may be broken arbitrarily):

```
IndSet(v, include) {
    if include == True then add v to S;
    for each u in children[v] {
        if include == True then IndSet(u, False);
        else IndSet(u, IN[u] >= OUT[u]);
    }
}

S = empty set;
IndSet(r, IN[r] >= OUT[r]);
return S;
```

One can check that the above algorithm maintains the choices dictated by the two recurrences in Theorem 1, thereby returning a maximum-weight independent set of T, and runs in time $\Theta(n)$.